

October 4, 2020

1 Programació per a *Data Science*

1.1 Unitat 3: Estructures de control i funcions en Python

1.2 Exercicis per practicar

Els següents 5 exercicis no puntuen per a la PAC, però us recomanem que els intenteu resoldre abans de passar als exercicis propis de la PAC. Trobareu les solucions a aquests exercicis al final del Notebook.

1.2.1 Exercici 1

L'exercici següent consisteix en passar un nombre en base 16 (hexadecimal, 0-9 / A-F) a base 10 (decimal). Per a això, heu de crear una **funció** que donat una *string* que representa un nombre en hexadecimal, per exemple, AE3F, retorni el nombre natural corresponent. En aquest cas el resultat seria 44607.

NM

```
[1]: # Resposta
```

1.2.2 Exercici 2

Escriure una funció que donat un nombre enter positiu, **n**, generi un arxiu de nom **output.txt** que contindrà **n** línies, on cada línia ha de mostrar un nombre consecutiu de lletres **A**. NM

Per exemple, si **N = 4**, l'arxiu generat tindrà els següent contingut:

```
A
AA
AAA
AAAA
```

```
[2]: # Resposta
```

1.2.3 Exercici 3

Completeu el codi necessari per calcular el nombre de vocals i de consonants respectivament d'un text. NM

```
[3]: def contar_vocals_y_consonants(text):
      # Contar les vocals i consonants contingudes en el string text
      num_vocals = 0
      num_consonants = 0

      # Codi a completar

      return (num_vocals, num_consonants)
```

```
[4]: # Resposta
```

1.2.4 Exercici 4

Escriu una funció que, acceptant argument de tipus llista, com \$llista = [a, b, c, ...] \$ **retorni una funció** que calculi el polinomi $f(x) = a + b \cdot x + c \cdot x^2 + \dots$. Hi ha moltes formes de resoldre aquestd exercici. Pots ajudar-te de la funció `eval()`, un `for` funcions lambda, `map/reduce` o la funció `zip()`. El

```
[ ]: # Respuesta
def creador(x):
    # Definició de la funció

f = creador([1,2,3,4])
f(3)
```

1.2.5 Exercici 5

Un nombre primer és aquell que només és divisible per ell mateix i per 1.

Escriu un codi que comprovi si un nombre `x = 15` és només divisible per 1 o per ell mateix. Escriu aquest codi usant un iterador (amb un `for` o `while`) que escombrï tots els valors des de 2 a `x-1`. Crea una variable, `divisible`, que tingui per defecte valor `False` i assigni el valor `True` si al llarg de la iteració troba un nombre natural divisible. Podeu fer servir l'operador mòdul `a % b` per saber si un nombre `b` és divisible per `a`.

NM

```
[ ]: # Resposta
```

1.3 Solucions als exercicis per practicar

1.3.1 Exercici 1

L'exercici següent consisteix en passar un nombre en base 16 (hexadecimal, 0-9 / A-F) a base 10 (decimal). Per a això, heu de crear una **funció** que donat una *string* que representa un nombre en hexadecimal, per exemple, `AE3F`, retorni el nombre natural corresponent. En aquest cas el resultat seria `44607`. **Resposta**

La formula para convertir un número hexadecimal a un número decimal, tomando como ejemplo el número AE3F, es: $A * 16^{**3} + E * 16^{**2} + 3 * 16^{**1} + F * 16^{**0} = 10 * 16^{**3} + 14 * 16^{**2} + 3 * 16^{**1} + 15 * 16^{**0}$

```
[1]: def hex_to_dec(nombre_hexadecimal):

    hex_mapping = {'0': 0, '1': 1, '2': 2, '3': 3, '4': 4, '5': 5,
                   '6': 6, '7': 7, '8': 8, '9': 9, 'A': 10, 'B': 11,
                   'C': 12, 'D': 13, 'E': 14, 'F': 15}

    # Primer , comprobem que sigui hexadecimal.
    all_hex = True
    for c in nombre_hexadecimal:
        if not c in hex_mapping.keys():
            all_hex = False

    if all_hex:

        # Definimos la base para realizar las operaciones
        base = 16
        nombre_decimal = 0

        # Invertim el nombre hexa porque sigui més fàcil treballar amb els
        ↪ índexs
        nombre_hexadecimal = nombre_hexadecimal[::-1]

        for i in range(len(nombre_hexadecimal)):
            # Per cada caràcter hexadecimal, apliquem la fórmula c * base ** i,
            # On c és la representació decimal del caràcter i
            # Sumem el resultat a el resultat obtingut en la iteració anterior
            nombre_decimal = nombre_decimal +
            ↪ hex_mapping[nombre_hexadecimal[i]] * base**i

        return nombre_decimal
    else:
        print("El nombre introduït no és correcte")

print(hex_to_dec('AE3F'))
print(hex_to_dec('FFF'))
print(hex_to_dec('123'))
```

44607

4095

291

1.3.2 Exercici 2

Escriure una funció que donat un nombre enter positiu, `n`, generi un arxiu de nom `output.txt` que contindrà `n` línies, on cada línia ha de mostrar un nombre consecutiu de lletres `A`. NM </ span>

Per exemple, si `N = 4`, l'arxiu generat tindrà els següent contingut:

```
A
AA
AAA
AAAA
```

Resposta

```
[18]: # Definim una funció que rebi un nombre enter de paràmetre
def generar_arxiu(N):
    # Obrim output.txt en mode escriptura.
    # El paràmetre 'w' farà que, si l'arxiu ja existeix, es sobreescriui
    with open('output.txt', 'w') as fd:

        # Recorrem el rang [0 ... N-1]
        for i in range(N):
            # Generem una cadena amb nombre ascendent de caràcters A
            linea = 'A' * (i + 1)
            # I ho escrivim al fitxer, junt amb un retorn de línia
            fd.write(linea + '\n')

generar_arxiu(4)
```

1.3.3 Exercici 3

Completeu el codi necessari per calcular el nombre de vocals i de consonants respectivament d'un text.

Resposta

```
[3]: def contar_vocals_y_consonants(text):
    # Conta vocals i consonants en una string text
    num_vocals = 0
    num_consonants = 0

    # Definim una llista amb les vocals
    vocals = ['a', 'e', 'i', 'o', 'u']

    for c in text.lower(): # Podem convertir el text a minúscules per
        ↪ simplificar el procés
        if c in vocals:
            num_vocals = num_vocals + 1
        elif c > 'a' and c <= 'z':
            num_consonants = num_consonants + 1
```

```

    return (num_vocals, num_consonants)

text = "Orbiting Earth in the spaceship, I saw how beautiful our planet is. \
       People, let us preserve and increase this beauty, not destroy it!"

num_vocals, num_consonants = contar_vocals_y_consonants(text)
print( "El nombre de vocals és %d." % num_vocals)
print( "El nombre de consonants és %d." % num_consonants)

```

El nombre de vocals és 44.
El nombre de consonants és 62.

1.3.4 Exercici 4

Hi ha moltes formes de resoldre aquest exercici. Pots ajudar-te de la funció `eval()`, un `for` funcions `lambda`, `map/reduce` o la funció `zip()`. El

```

[9]: # Resposta

# Solució en base a un for
def creador(lista):
    # Definició de la funció
    def pol(x):
        toeval = ""
        for coef,expo in enumerate(lista): #enumerate ens torna el coeficient i
        ↪ l'ordre en la llista
            toeval += '+' + str(expo) + "*x**"+str(coef) + ' '
        print(toeval)
        return(eval(toeval))
    return(pol) # retornem la funció

f = creador([1,2,3,4])
f(3)

```

+1*x**0 +2*x**1 +3*x**2 +4*x**3

[9]: 142

```

[7]: # Resposta
from functools import reduce # cal importar reduce
def creador(llista):
    # Aquesta solució usa map/reduce/zip eper comptes d'un for
    def concat(a,b): #funció para concatenar termes
        return(a+' '+b)
    def element(t): # funció para construir un terme del polinomi c*x^d
        return( str(t[0])+"*x**"+str(t[1]) )
    def pol(x):

```

```

        toeval= reduce(concat,map(element,
                                zip(llista,range(len(llista))))) # zip torna tuples
→ amb parelles d'elements de les llistes
        print(toeval)
        return(eval(toeval))
    return(pol)

f = creador([1,2,3,4])
f(3)

```

$1*x**0 + 2*x**1 + 3*x**2 + 4*x**3$

[7]: 142

```

[10]: # Resposta
# Una versió más compacte con definicions lambda
from functools import reduce # cal importar reduce
def creador(llista):
    # Definició de la funció
    def pol(x):
        out = eval(reduce(lambda x,y: x+'+'+y,
                        map(lambda x: str(x[0])+"*x**"+str(x[1]),
                            zip(llista,range(len(llista)))))
        return(out)
    return(pol)

f = creador([1,2,3,4])
f(3)

```

[10]: 142

1.3.5 Exercici 5

Un nombre primer és aquell que només és divisible per ell mateix i per 1.

Escriu un codi que comprovi si un nombre $x = 15$ és només divisible per 1 o per ell mateix. Escriu aquest codi usant un iterador (amb un for o while) que escombri tots els valors des de 2 a $x-1$. Crea una variable, `divisible`, que tingui per defecte valor `False` i assigni el valor `True` si al llarg de la iteració troba un nombre natural divisible. Podeu fer servir l'operador mòdul `a % b` per saber si un nombre `b` és divisible per `a`. NM

```

[ ]: # Definim primer x com 15 i la etiqueta divisible

x = 15

divisible = False

# Creem un iterador des de 2 a x-1 (range (2, x) on x no estarà inclòs)

```

```
# Mirem, per a cada valor de l'iterador, si és divisible amb  
# la funció mòdul%, si és així, la funció mòdul  
# ens tornarà 0 i assignarem divisible a True  
# En aquest cas el nombre no serà primer  
  
for i in range(2,x):  
    if not x%i:  
        divisible = True  
  
print(divisible)
```